Proyecto de Ingeniería de Datos y Privacidad por Diseño

UrbanEats:

Plataforma de delivery y analítica de cadena alimentaria con privacidad por diseño

\

Presentado por: Ronaldo Manuel Barrios Garrido

Fecha de Entrega: 03 de Marzo de 2026

Asignatura: Fundamentos de la Tecnología Datos Masivos

Profesor/Evaluador: José Fernández Tamames

Mensaje para el CEO (Elevator Pitch): *"En UrbanEats hemos transformado nuestra mayor crisis en una ventaja competitiva. He rediseñado toda nuestra plataforma de datos con un objetivo claro: reconstruir la confianza de nuestros usuarios y blindarnos ante futuras multas de la AEPD. A partir de ahora, ya no guardaremos historiales que expongan la religión o la salud de nuestros clientes; nos quedaremos solo con lo que da dinero. Con este nuevo sistema, sabremos exactamente en qué zonas faltan repartidores para evitar retrasos, detectaremos comida fría antes de que el cliente se queje, y blindaremos nuestros algoritmos frente a la competencia. En resumen: entregas más rápidas, cero multas y mayor rentabilidad por cada pedido."*

Paso 1 - Inventario de fuentes (Mapa de datos)

Para tener una visión clara de nuestra infraestructura y los riesgos asociados tras el incidente con la AEPD, he clasificado las 13 fuentes de datos de nuestro ecosistema. He categorizado la latencia de las fuentes de scraping como "Batch" por su naturaleza de ejecución programada.

Fuente	Estructura	Latencia	Propiedad	Sensibilidad
1. orders.csv	Estructurado	Batch	Propietario	Contiene PII y datos sensibles inferibles (dirección de entrega + items_json).
2. payments.csv	Estructurado	Batch	Propietario	Contiene PII indirecta (token de tarjeta, order_id).
3. restaurants.csv	Estructurado	Batch	Propietario	No contiene PII.
4. riders_shifts.csv	Estructurado	Batch	Propietario	Contiene PII indirecta (rider_id).
5. app_events.jsonl	Semiestructurado	Near real-time	Propietario	Contiene PII (user_id).
6. rider_telemetry.jsonl	Semiestructurado	Streaming (3s)	Propietario	Contiene PII (ubicación constante del trabajador).
7. bag_sensors.jsonl	Semiestructurado	Streaming (10s)	Propietario	Contiene PII indirecta (asociable al rider_id).

Fuente	Estructura	Latencia	Propiedad	Sensibilidad
8. support_tickets.json	No estructurado	Batch	Propietario	Contiene PII accidental en texto libre.
9. chat_transcripts.txt	No estructurado	Batch	Propietario	Contiene PII accidental en conversaciones.
10. reviews.json	No estructurado	Near real-time	Propietario / Tercero	Contiene PII accidental y datos sensibles inferibles.
11. Web reg. sanitarios	No estructurado	Scraping (Batch)	Tercero	No contiene PII (datos públicos de negocios).
12. Web competidores	No estructurado	Scraping (Batch)	Tercero	No contiene PII.
13. Open data municipal	Estructurado	API/CSV (Batch)	Tercero	No contiene PII.

Identificación de elementos críticos:

Como pide el CEO, tenemos que equilibrar el negocio con la seguridad. Por ello, destaco lo siguiente:

- **Las 3 fuentes más críticas para el negocio:**
    1. **orders.csv**: Es el *core* transaccional. Sin él no hay cálculo de ingresos ni métricas base.
    2. **rider_telemetry.jsonl**: Absolutamente vital para el objetivo de reducir el tiempo medio de entrega mediante una asignación eficiente.
    3. **app_events.jsonl**: Clave para entender la experiencia del usuario, detectar caídas de conversión y mejorar la app.

- **Las 3 fuentes más sensibles para la privacidad:**

1. **orders.csv**: Es nuestra principal amenaza reputacional. Cruzar la dirección con el contenido del pedido (items_json) permite deducir religión o estado de salud (art. 9 RGPD).
2. **support_tickets.json / chat_transcripts.txt**: Un foco altísimo de "PII accidental" difícil de gobernar, ya que los usuarios introducen datos personales no solicitados en texto libre.
3. **rider_telemetry.jsonl**: Monitorizar la geolocalización de una persona cada 3 segundos supone un riesgo altísimo de vigilancia intensiva y requiere una sólida justificación de necesidad y proporcionalidad.

Paso 2 - Selección de datos útiles

Como responsable de este rediseño, tengo muy claro que no podemos almacenar ni procesar todo por defecto. El incidente de la AEPD nos enseñó que acumular datos innecesarios es un pasivo tóxico. Por ello, he filtrado estrictamente qué campos usamos y cuáles descartamos para cada objetivo, basándome en el equilibrio coste-beneficio y el riesgo de sesgo/privacidad.

1. Objetivo: Reducir tiempo medio de entrega (optimización logística)

- **Se usa:** order_id, order_ts, delivery_ts, restaurant_id, rider_id (de orders.csv). De la telemetría usamos lat, lon, speed_kmh , y de los restaurantes avg_prep_time_min. También integramos zonas de tráfico y obras del Open Data municipal.
- **No se usa:** customer_id y delivery_address (dirección postal exacta).
- **Justificación:** Para calcular la ruta óptima y asignar el pedido, el algoritmo solo necesita saber de qué coordenadas sale la comida y a qué coordenadas aproximadas (o zona/código postal) va. Usar la dirección exacta o identificar al cliente en el modelo logístico no aporta valor al algoritmo y genera un riesgo de reidentificación innecesario.

2. Objetivo: Gestión de incidencias y satisfacción del cliente

- **Se usa:** event_type, error_code (de app_events.jsonl), temp_celsius, lid_open (de bag_sensors.jsonl), rating (de reviews.json) y health_score (registros sanitarios scrapeados).
- **No se usa:** El texto libre de las reseñas sin procesar, ni el rider_id para análisis agregados de satisfacción.
- **Justificación (Coste-beneficio y sesgos):** Medir la temperatura de la bolsa nos da la "salud" del pedido objetivamente. Descartar la identidad del rider en la vista macro evita sesgos contra trabajadores individuales. Los textos crudos se descartan aquí para evitar que PII accidental contamine los cuadros de mando de satisfacción global.

3. Objetivo: Análisis de ingresos y comisiones por segmento

- **Se usa:** total_price, city_id (de orders.csv), amount, tip_amount (de payments.csv), cuisine_type (de restaurants.csv), y las tarifas de competidores (Glovo, Uber Eats).
- **No se usa:** items_json (el contenido exacto del pedido), ni card_token_last4.
- **Justificación:** Al CEO y a Finanzas les interesan las ventas por tipo de cocina y ciudad. El cruce de items_json con direcciones fue exactamente lo que provocó la fuga masiva , permitiendo inferir creencias religiosas o condiciones médicas (datos muy sensibles). Descartar items_json en la capa analítica financiera tiene coste cero para el negocio y un beneficio inmenso en cumplimiento RGPD.

4. Objetivo: Operación de soporte (búsqueda en tickets/chat/reseñas)

- **Se usa:** ticket_id, category, priority (de support_tickets.json). Los textos libres (body, transcripciones de chat, reseñas) **solo se usarán** tras aplicarles un filtro de ofuscación.
- **No se usa:** Textos en formato "raw" (crudo) directos de la fuente.
- **Justificación:** El equipo de soporte necesita leer los chats para resolver problemas , pero los clientes escriben nombres y teléfonos sin control (PII accidental). La regla de diseño es indexar los textos en el buscador para encontrar patrones (ej. "llegó frío"), pero enmascarando previamente con técnicas NLP los datos personales para que los agentes de Nivel 1 no vean información que no necesitan.

Paso 3 - Reglas de calidad

He revisado los datos a fondo y he detectado las "trampas" deliberadas que mermarían la confianza en nuestros KPIs. Si el CEO va a tomar decisiones sobre los tiempos de entrega, no podemos tener sensores reportando −15°C o pedidos con precios negativos.

Aquí presento mi tabla con las 8 reglas de calidad fundamentales que implementaré en el pipeline (entre la *Raw Zone* y la *Curated Zone*) para sanear la información y garantizar la privacidad:

#	Fuente afectada y campo	Dimensión de calidad	Regla concreta a evaluar	Método de medición	Acción ante incumplimiento
1	orders.csv order_id	**Unicidad**	COUNT(order_id) == 1 en una ventana temporal de 24h.	Query SQL (agrupación y conteo por order_id).	**Corregir / Descartar:** Eliminar el duplicado, conservando únicamente la fila con el timestamp de actualización más reciente.

#	Fuente afectada y campo	Dimensión de calidad	Regla concreta a evaluar	Método de medición	Acción ante incumplimiento
2	bag_sensors.jsonl temp_celsius	**Validez**	temp_celsius BETWEEN -5 AND 60 (limitar valores absurdos).	Script de filtrado en el flujo de streaming (ej. Spark Streaming).	**Descartar:** Ignorar la lectura puntual por asumirse fallo del sensor IoT (glitch de hardware).
3	rider_telemetry.jsonl lat, lon	**Exactitud**	Las coordenadas deben estar dentro del polígono geográfico de la ciudad asignada (city_id).	Script geoespacial / PostGIS (ST_Contains).	**Descartar:** Filtrar el punto anómalo (salto de GPS) para no alterar la velocidad media ni la ruta.
4	orders.csv total_price	**Validez**	total_price >= 0	Alerta automática en la carga Batch / CHECK en SQL.	**Cuarentena:** Aislar en tabla de errores y alertar a Soporte/Pagos, podría ser un bug de aplicación de códigos promocionales.
5	orders.csv order_ts, delivery_ts	**Consistencia / Comparabilidad**	delivery_ts (convertido a UTC) > order_ts (en UTC).	Script de transformación (ETL en Python/SQL) con gestión de zonas horarias.	**Corregir:** Normalizar todos los timestamps a UTC (tz_convert) antes de guardarlos en la capa *Curated*.
6	rider_telemetry.jsonl rider_id	**Consistencia (Integridad Referencial)**	El rider_id transmitiendo debe existir en la tabla de turnos (riders_shifts.csv).	Query de validación cruzando eventos de telemetría vs. tabla de turnos activa.	**Cuarentena:** Generar alerta de seguridad. Podría tratarse de un rider no autorizado ("pirata") o un fallo de sincronización de turnos.

#	Fuente afectada y campo	Dimensión de calidad	Regla concreta a evaluar	Método de medición	Acción ante incumplimiento
7	orders.csv restaurant_id	**Consistencia (Integridad Referencial)**	restaurant_id debe existir en el catálogo maestro (restaurants.csv).	Query SQL (LEFT JOIN para detectar huérfanos).	**Cuarentena:** Aislar el pedido. Probablemente la dimensión de restaurantes esté desactualizada en el *Warehouse*.
8	support_tickets.json y reviews.json body, comment	**Validez (Cumplimiento Privacidad)**	El texto no debe contener patrones regex que coincidan con teléfonos, DNI o emails.	Script de NLP/Expresiones regulares (DLP) ejecutado en la ingesta.	**Corregir:** Enmascarar la información inyectando la etiqueta [PII_REMOVED] antes de almacenar el texto para indexación.

Mi enfoque de negocio: Como indico en la Regla 8, sanear los datos no es solo cuestión de "limpiar nulos" o arreglar tipos de datos; es nuestra primera y más barata línea de defensa contra multas de la AEPD. Si evitamos que un número de teléfono o un error de GPS entre al Data Warehouse, nos ahorramos problemas de privacidad y decisiones operativas equivocadas.

Paso 4 - Diseño de ingesta (batch + stream + scraping)

Para diseñar este pipeline, mi prioridad ha sido crear un flujo robusto que separe claramente la adquisición de datos de su consumo, aplicando el principio de "privacidad por diseño" desde el minuto uno. He estructurado la arquitectura en 4 capas lógicas para evitar cuellos de botella y garantizar que el CEO y Operaciones tengan datos fiables.

Aquí presento el flujo completo *End-to-End*:

1. Capa de Ingesta (Mecanismos por fuente)

He asignado el patrón de ingesta según la latencia que exige el negocio y el riesgo de la fuente:

- **Batch (Diario/Horario):** orders.csv, payments.csv, restaurants.csv, riders_shifts.csv se extraen mediante ETL batch desde nuestras bases transaccionales. support_tickets.json y

chat_transcripts.txt se ingieren por batch pero pasando por una función de ofuscación (DLP) *en vuelo* para enmascarar la PII accidental antes de tocar disco.

- **Near Real-Time (Micro-batch / Webhook):** app_events.jsonl y reviews.json. Necesitamos reaccionar rápido a caídas en la app o malas reseñas.
- **Streaming:** rider_telemetry.jsonl y bag_sensors.jsonl se inyectan a través de un bus de eventos (ej. Apache Kafka).
- **Scraping / API Poll:** Webs de competidores, registros sanitarios (Scraping) y Open Data municipal (API).

2. Raw Zone (Data Lake)

Es nuestra zona de aterrizaje inmutable. Aquí guardamos los datos tal cual llegan (formato original CSV, JSON, HTML) para garantizar la auditabilidad y la reproductibilidad. La única excepción es la PII accidental en textos libres, que ya llega enmascarada desde la ingesta por precaución legal.

3. Curated Zone (Data Lakehouse)

Aquí es donde ocurre la "magia" y aplicamos las 8 reglas de calidad del Paso 3. Transformamos todo a un formato columnar eficiente (ej. Parquet). Normalizamos los *timestamps* a UTC (delivery_ts > order_ts) , filtramos las coordenadas imposibles del GPS de los riders y eliminamos los order_id duplicados. Particionamos los datos por city_id y fecha para optimizar costes de lectura.

4. Serving Zone

Los datos ya limpios y enriquecidos se distribuyen a sus sistemas finales:

- El **Data Warehouse (SQL)** lee los datos agregados para los KPIs financieros del CEO.
- El **Almacén Operacional (NoSQL)** recibe el estado en tiempo real de riders y pedidos.
- El **Motor de Búsqueda** indexa los tickets y chats (ya sin PII) para el equipo de soporte.

Requisitos Técnicos Específicos

A. Estrategia de Scraping/Crawling (Webs competidores y Registros Sanitarios)

- **Ejecución:** Programada (ej. domingos a las 3:00 AM) para no saturar recursos.
- **Politeness y Ética:** Mi scraper leerá primero el archivo robots.txt para asegurar que no tocamos rutas prohibidas. Implementaré retardos aleatorios (*delays* de 5 a 15 segundos) entre peticiones para simular tráfico humano y no tumbar los servidores de Glovo o del Ayuntamiento (mitigando riesgos legales).
- **Normalización:** Guardaremos el código HTML en crudo en la *Raw Zone* (como evidencia ante posibles disputas). Luego, un proceso de parseo (ej. BeautifulSoup) extraerá las

tablas (tarifas, puntuación de sanidad) y las convertirá en formato estructurado (CSV/Parquet) para la *Curated Zone*.

B. Gestión del Streaming Dual (Telemetría a 3s vs. Sensores a 10s) Tenemos un conflicto de cadencias: la posición del rider (3s) es crítica para asignar pedidos rápido, mientras que la temperatura (10s) es importante pero no debe bloquear el flujo principal.

- **Mi solución:** Utilizaré dos *topics* separados en nuestro bus de mensajería (ej. Kafka). El topic de telemetría tendrá alta prioridad y particionado por ciudad para asegurar latencia sub-segundo. El topic de sensores se procesará de forma independiente.
- **Integración:** En la *Curated Zone*, utilizaré una técnica de procesamiento de flujos cruzados (*Window Join*). Asociaré la temperatura de la bolsa de los últimos 10 segundos a la posición del rider en ese intervalo temporal, consiguiendo tener la foto completa sin penalizar el rendimiento del algoritmo de asignación logística.

Paso 5 - Arquitectura de almacenamiento

Para este paso, he diseñado una arquitectura de almacenamiento pragmática y orientada a resolver nuestros dos grandes frentes: dar soporte a la operativa en tiempo real (que la app no se cuelgue y asigne riders rápido) y garantizar que el CEO tenga sus analíticas sin exponer datos personales, asegurando el cumplimiento normativo.

A continuación, presento mi diseño estructurado en los 4 componentes exigidos:

Diagrama Lógico de la Arquitectura (Bloques y Flujos)

```
[FUENTES DE DATOS] (Streaming 3s/10s, Batch, Scraping)
       |
       v
+---------------------------------------------------+
| 1. DATA LAKE (Raw Zone -> Curated Zone)           |
| Función: Auditoría, ingesta masiva y archivo.     |
+---------------------------------------------------+
       | (Procesamiento / Limpieza / Ofuscación DLP)
       |
       +--------------------+-----------------------+
       |                    |                       |
       v                    v                       v
+---------------+   +----------------+   +-------------------+
| 2. ALMACÉN    |   | 3. DATA        |   | 4. MOTOR DE       |
| OPERACIONAL   |   | WAREHOUSE      |   | BÚSQUEDA          |
| (NoSQL)       |   | (SQL)          |   | (Inverted Index)  |
|               |   |                |   |                   |
| Operativa App |   | Analítica/BI   |   | Soporte al cliente|
+---------------+   +----------------+   +-------------------+
       |                    |                       |
       v                    v                       v
 [Asignación Riders]  [Dashboards CEO]     [Panel Agentes CS]
```

Detalle de los Componentes

1. Data Lake (Raw + Curated)

- **Función:** Es nuestro "notario". Almacena de forma inmutable la historia completa para auditorías de la AEPD y permite reprocesar la información si cambiamos la lógica de negocio.
- **Qué almacena:** Todo. Pedidos, telemetría cruda, logs de tickets, y el HTML descargado del scraping.
- **Formato:** JSON/CSV/HTML en la capa *Raw*; Parquet (columnar optimizado) en la capa *Curated*.
- **Retención:** Larga. 5 años para datos transaccionales (obligación fiscal), pero aplicando pseudoanonimización a los 30 días en la capa *Curated* para cumplir con la minimización de datos.
- **Patrón de acceso:** Escrituras masivas continuas (Append-only) y lecturas infrecuentes de gran volumen (por procesos Batch/ETL).

2. Data Warehouse (SQL)

- **Función:** Analítica y Business Intelligence. Aquí es donde Finanzas y el CEO consultan los KPIs de ingresos, tiempos de entrega y satisfacción.
- **Qué almacena:** Datos agregados y limpios. Pedidos (¡sin el detalle items_json!), pagos consolidados, y medias de tiempos por zona/franja horaria.
- **Formato:** Relacional (Esquema en Estrella optimizado para consultas analíticas).
- **Retención:** 2 a 3 años para permitir comparativas interanuales (YoY).
- **Patrón de acceso:** OLAP (Online Analytical Processing). Lecturas analíticas complejas (JOINs pesados, GROUP BY) ejecutadas por herramientas de BI y analistas.

3. Almacén Operacional (NoSQL)

- **Función:** Es el motor en tiempo real que sostiene la App. Gestiona la asignación y el seguimiento activo.
- **Qué almacena:** El estado *actual* (no el histórico). Posición última del rider, pedido en curso, estado de la bolsa térmica y batería de la moto.
- **Formato:** Documentos JSON (ej. MongoDB) o Clave-Valor (ej. Redis) para máxima velocidad.
- **Retención:** Muy corta. Solo mientras el pedido esté activo o el rider en turno (aprox. 24-48 horas por si hay reclamaciones inmediatas). Luego, el dato "muere" aquí (porque ya está respaldado en el Data Lake).
- **Patrón de acceso:** OLTP (Online Transaction Processing). Miles de lecturas/escrituras muy rápidas y precisas (búsqueda por rider_id u order_id).

4. Motor de Búsqueda

- **Función:** Herramienta rápida e intuitiva para que el equipo de atención al cliente detecte patrones y resuelva quejas.
- **Qué almacena:** Textos de tickets, chats y reseñas indexados. **Nota de negocio/privacidad:** Solo almacena los textos *después* de que nuestro pipeline en la ingesta haya ofuscado la PII (DNI, teléfonos, nombres).
- **Formato:** Índices invertidos (ej. Elasticsearch/OpenSearch).
- **Retención:** 1 a 2 años, útil para el entrenamiento de bots de soporte y análisis de tendencias de reclamaciones a medio plazo.
- **Patrón de acceso:** Búsquedas *Full-Text*, difusas (fuzzy search) y agregaciones lingüísticas a muy alta velocidad.

He priorizado una solución coherente donde cada pieza tiene un propósito claro, separando la carga operativa de la analítica, lo cual nos evitará bloqueos en el sistema.

Paso 6 - Decisión CAP (obligatoria para 2 subsistemas)

Para diseñar estos dos subsistemas de UrbanEats, he aplicado el Teorema CAP partiendo de una premisa clara: en sistemas distribuidos, la partición de red (P) va a ocurrir tarde o temprano. La decisión crítica es qué sacrificamos cuando eso pasa.

Subsistema A — Asignación de pedido a rider (tiempo real)

- **Decisión:** Elijo **AP (Disponibilidad y Tolerancia a Particiones)**. Priorizo la Disponibilidad frente a la Consistencia estricta.
- **Escenario de partición:** Imaginemos un viernes por la noche en Madrid; el clúster de asignación pierde conexión temporal con la base de datos central. Si un cliente hace un pedido, la aplicación sigue aceptándolo y el nodo local asigna el pedido al rider "Juan" por proximidad. Simultáneamente, otro nodo incomunicado asigna a "Juan" otro pedido distinto.
- **Justificación de negocio:** Como negocio de *delivery*, prefiero generar una pequeña inconsistencia temporal a que la app muestre un error y el usuario se vaya a la competencia (Glovo o Uber Eats). Necesitamos asignar en menos de 10 segundos.
- **Mitigación de la propiedad sacrificada (Consistencia):** Para recuperar la consistencia (logrando *consistencia eventual*), implemento un sistema de *leases* (bloqueos temporales en la app del rider) y colas de reasignación. Cuando la red se restablece, el sistema detecta que Juan tiene dos pedidos en conflicto. La app de Juan solo le permite confirmar el primero que le llegó a su pantalla; el segundo pedido entra instantáneamente a una cola de reasignación para enviarse al siguiente rider más cercano de forma transparente para el cliente.

Subsistema B — Dashboard de ingresos y KPIs (analítica)

- **Decisión:** Elijo **CP (Consistencia y Tolerancia a Particiones)**. Priorizo la Consistencia absoluta frente a la Disponibilidad.
- **Escenario y justificación:** Este cuadro de mando se actualiza cada noche con datos contables de ingresos y comisiones. Si durante la carga nocturna hay una caída de red entre el Data Lake y el Data Warehouse, prefiero que el proceso falle y el dashboard no esté disponible a primera hora (o muestre los datos del día anterior), a que el CEO vea datos financieros incompletos o duplicados. En finanzas, un dato erróneo es mucho peor que un dato retrasado.
- **Garantía de fiabilidad:** Garantizo la consistencia asegurando que el pipeline de datos batch sea estrictamente *idempotente* y utilice transacciones ACID en el modelo SQL. Si hay partición, el trabajo de carga hace *rollback*. Al volver la red, el sistema lo reintenta de cero sin duplicar ningún ingreso.
- **Diferencia de requisitos:** Los requisitos son opuestos al Subsistema A porque la naturaleza del usuario cambia. En el Subsistema A, el cliente final requiere inmediatez impulsiva para comprar ; en el Subsistema B, la dirección financiera requiere exactitud absoluta para auditar el negocio, pudiendo tolerar esperar un par de horas si hay un fallo nocturno.

Paso 7 - Modelado SQL (analítico)

Para que nuestro CEO y el equipo financiero puedan analizar el negocio de forma rápida y sin poner en riesgo la privacidad de los usuarios, he diseñado un modelo dimensional en estrella (Star Schema). Es sencillo, coherente y, lo más importante, no contiene ni un solo dato personal o inferible (como pedía el diseño de privacidad).

El "grano" de la tabla de hechos es **un pedido completado**.

1. Diagrama de Tablas (Esquema en Estrella)

Tabla de Hechos:

- **fact_orders**
  - *PK:* order_id
  - *FKs:* date_id, time_id, city_id, restaurant_id
  - *Métricas:* total_price (numeric), commission_amount (numeric), delivery_time_mins (integer, precalculado en la capa Curated), has_incident (boolean/1-0, cruzado desde tickets), rating (integer 1-5, cruzado desde reviews).
  - *Nota de Privacidad:* Como decidí en el Paso 2, aquí NO incluimos items_json ni customer_id.

Tablas de Dimensiones:

- **dim_date**: date_id (PK), date, year, month, is_weekend.
- **dim_time**: time_id (PK), hour, time_band (ej. 'Desayuno', 'Almuerzo', 'Cena').
- **dim_location**: city_id (PK), city_name, zone.
- **dim_restaurant**: restaurant_id (PK), name, cuisine_type.

2. Consultas SQL de Ejemplo para KPIs Estratégicos

Aquí presento 3 consultas directas contra este modelo para extraer los KPIs que nos pide la dirección:

Consulta 1: Ingresos por ciudad y zona en el último mes Esta query le dirá a finanzas dónde estamos ganando dinero.

SQL

```
SELECT
   l.city_name,
   l.zone,
   SUM(f.total_price) AS ingresos_totales,
   SUM(f.commission_amount) AS comisiones_totales
FROM fact_orders f
JOIN dim_location l ON f.city_id = l.city_id
JOIN dim_date d ON f.date_id = d.date_id
WHERE d.month = EXTRACT(MONTH FROM CURRENT_DATE)
  AND d.year = EXTRACT(YEAR FROM CURRENT_DATE)
GROUP BY 1, 2
ORDER BY ingresos_totales DESC;
```

Consulta 2: Tiempo medio de entrega por franja horaria Vital para que operaciones entienda si colapsamos durante las cenas.

SQL

```
SELECT
   t.time_band,
   AVG(f.delivery_time_mins) AS tiempo_medio_entrega_minutos
FROM fact_orders f
JOIN dim_time t ON f.time_id = t.time_id
GROUP BY 1
ORDER BY tiempo_medio_entrega_minutos DESC;
```

Consulta 3: Tasa de incidencias y satisfacción media por restaurante Para que el equipo de calidad penalice o premie a los restaurantes.

SQL

```
SELECT
   r.name AS restaurante,
   r.cuisine_type,
   (SUM(CAST(f.has_incident AS INT)) * 100.0 / COUNT(f.order_id)) AS tasa_incidencias_pct,
   AVG(f.rating) AS satisfaccion_media
FROM fact_orders f
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
GROUP BY 1, 2
HAVING COUNT(f.order_id) > 50 -- Filtramos restaurantes con poco volumen para no sesgar
ORDER BY tasa_incidencias_pct DESC;
```

Con este modelo tenemos cubierto el análisis histórico y agregado con una alta rentabilidad de negocio y cero exposición de PII.

Paso 8 - Modelado NoSQL (operacional / tiempo real)

Para sostener la app móvil y el sistema de asignación logística en tiempo real, he diseñado un modelo NoSQL orientado a documentos (tipo MongoDB). El objetivo aquí es la latencia mínima: lecturas y escrituras ultrarrápidas. He separado la información en dos colecciones principales para evitar bloqueos por actualizaciones constantes.

Aquí presento los documentos JSON y la estrategia de modelado:

1. Documentos JSON de Ejemplo

Colección A: active_riders (Estado del rider) Esta colección se actualiza constantemente (cada 3s con la telemetría).

JSON

```
{
  "_id": "rider_8472",
  "status": "DELIVERING",
  "active_order_id": "ord_99381",
  "current_location": {
    "type": "Point",
    "coordinates": [-3.703790, 40.416775]
  },
  "speed_kmh": 22.5,
  "vehicle": {
    "type": "e-moto",
    "battery_pct": 85
  },
  "bag_status": {
    "temp_celsius": 18.2,
    "lid_open": false,
    "last_updated": "2026-02-27T15:25:10Z"
  },
  "last_telemetry_ts": "2026-02-27T15:25:15Z"
}
```

Colección B: active_orders (Pedido activo) Esta colección mantiene el estado del pedido desde que se crea hasta que se entrega.

JSON

```
{
  "_id": "ord_99381",
  "status": "PICKED_UP",
  "assigned_rider_id": "rider_8472",
  "restaurant": {
    "id": "rest_445",
    "name": "Burger Central"
  },
  "customer": {
    "id": "cust_1029",
    "delivery_zone": "28013",
    "delivery_lat_lon_obfuscated": [-3.705000, 40.420000]
  },
  "eta_mins": 12,
  "event_history": [
    {"event": "CREATED", "ts": "2026-02-27T15:10:00Z"},
    {"event": "ACCEPTED_BY_RIDER", "ts": "2026-02-27T15:10:08Z"},
    {"event": "PICKED_UP", "ts": "2026-02-27T15:20:30Z"}
  ]
}
```

Nota de Privacidad (Privacy by Design): Fíjate que en customer **no guardo la dirección postal exacta** en el almacén operacional, solo la zona/código postal y coordenadas ofuscadas para el cálculo de ETA. Si un atacante entra a esta base de datos en tiempo real, no encontrará dónde vive exactamente el cliente.

2. Claves, Índices y Relaciones

- **¿Embeber o Referenciar?** He decidido **referenciar** el rider y el pedido (usando active_order_id y assigned_rider_id).
    - *Justificación de negocio/técnica:* Si embebiéramos el estado del rider dentro del pedido, tendríamos que reescribir todo el documento del pedido cada 3 segundos cuando llega una nueva coordenada GPS. Al referenciar, mantenemos las altísimas tasas de escritura de telemetría aisladas en active_riders, mientras que active_orders solo se actualiza cuando hay un cambio de estado real (ej. de "PREPARING" a "PICKED_UP").
- **Embebido aplicado:** Por el contrario, sí he **embebido** el historial de eventos (event_history) dentro del pedido y el estado de la bolsa (bag_status) dentro del rider, ya que son datos que pertenecen intrínsecamente a su entidad padre y se consultan siempre juntos.
- **Índices propuestos:**

1. Índice Geoespacial (2dsphere) en active_riders.current_location: Vital para buscar los riders más cercanos en milisegundos durante la asignación.
2. Índice B-Tree en active_orders.assigned_rider_id: Para búsquedas rápidas bidireccionales.

3. Consultas Típicas (Operacionales)

Consulta 1: Buscar riders disponibles cerca del restaurante (Asignación Logística) Esta consulta la ejecuta nuestro algoritmo del Paso 6 para asignar un pedido nuevo:

JSON
```
{
  "find": "active_riders",
  "filter": {
    "status": "AVAILABLE",
    "current_location": {
      "$near": {
        "$geometry": {
          "type": "Point",
          "coordinates": [-3.701, 40.415]
        },
        "$maxDistance": 2000
      }
    }
  },
  "limit": 5
}
```

Consulta 2: Cargar la pantalla de seguimiento de la App del cliente Cuando el cliente abre la app para ver por dónde va su comida:

Paso 1: Obtener el pedido activo:

JSON

```
{
  "find": "active_orders",
  "filter": {
    "_id": "ord_99381"
  },
  "limit": 1
}
```

Paso 2: Obtener la posición del rider asignado (usando el ID obtenido en el paso anterior) aplicando una proyección para no traer datos de más:

```
{
  "find": "active_riders",
  "filter": {
    "_id": "rider_8472"
  },
  "projection": {
    "current_location": 1,
    "eta_mins": 1,
    "bag_status": 1
  },
  "limit": 1
}
```

Con esto, el sistema operacional es robusto, rápido y seguro.

Paso 9 - Indexación y recuperación (tickets/chat/reseñas)

Para que el equipo de atención al cliente (Soporte) pueda apagar incendios rápidamente y detectar patrones sin violar la privacidad de los usuarios, he diseñado un motor de búsqueda unificado (tipo Elasticsearch / OpenSearch). Parto de la premisa de que los textos ya han sido "limpiados" de PII en la capa de ingesta (Paso 4) mediante técnicas de DLP.

1. Diseño del Índice y Normalización Lingüística

He unificado las tres fuentes (support_tickets, chat_transcripts y reviews) en un único índice lógico llamado customer_interactions.

- **Campos indexados:** interaction_type (ticket/chat/review), source (app/Google/TripAdvisor), subject, body_text, rating, priority, created_at.
- **Normalización lingüística (Pipeline de análisis):** Al operar en zonas turísticas (como Palma o Barcelona), recibimos quejas en varios idiomas.
    - **Tokenización:** Estándar (división por espacios y puntuación).
    - **Filtros (Lowercasing & Stopwords):** Eliminamos palabras vacías ("el", "la", "the", "and") usando diccionarios duales Español/Inglés.
    - **Stemming:** Aplicamos un *Snowball stemmer* para reducir las palabras a su raíz (ej. "frío", "fría", "friísimo" -> "fri"; "returned", "returning" -> "return") para maximizar el *recall* (recuperación).

2. Extracción de Entidades (NER - Named Entity Recognition)

No basta con buscar texto libre. Durante la ingesta, aplicaremos modelos de extracción (o diccionarios predefinidos) para etiquetar estructuradamente el texto no estructurado:

- **Entidades extraídas:** zone/neighborhood (barrios), restaurant_name, cuisine_type, error_code (ej. HTTP 500), device_os (iOS/Android). Estas entidades se guardan como campos keyword para poder filtrar y agrupar con exactitud.

3. Criterio de Ranking (Relevancia)

- **Algoritmo base:** Utilizaré **BM25** (evolución del TF-IDF), ya que penaliza mejor los textos excesivamente largos (como un chat interminable), dando resultados más precisos.
- **Boost (Pesos):** Le daré un multiplicador (boost: 3.0) al campo subject (el título del ticket) frente al body_text (boost: 1.0), ya que el título suele contener la intención principal del cliente. Además, aplicaré un *Decay Function* (recency boost) para que las interacciones de las últimas 24 horas aparezcan más arriba que quejas de hace 6 meses.

4. 5 Consultas Operativas Concretas

Aquí presento las consultas que usarían los agentes o los analistas de calidad (expresadas en lógica de búsqueda DSL / Lógica de filtrado):

1. **Top 10 motivos de reclamación en el último mes:**
    - *Tipo:* Agregación de términos (Terms Aggregation).
    - *Lógica:* Filtrar created_at >= now-30d y interaction_type = 'ticket'. Agrupar por el campo extraído category o error_code y devolver el TOP 10 ordenado por recuento de documentos.
2. **Reseñas con mención de “frío” o “cold” en pedidos de sushi en Madrid:**
    - *Tipo:* Búsqueda *Full-Text* + Filtros duros.
    - *Lógica:* match en body_text para ("frío" OR "cold"). Filtros exactos (filter context): cuisine_type = 'Sushi' AND city_id = 'Madrid'.
3. **Tickets donde el cliente menciona “devolución” con prioridad alta:**

- *Tipo:* Búsqueda *Full-Text* + Filtro.
- *Lógica:* match en body_text para la raíz de ("devolver", "devolución", "reembolso") con filtro exacto priority = 'HIGH'.

4. *(Mi propuesta 1)* **Detección de "bugs" críticos en la App móvil:**
    - *Tipo:* Búsqueda cruzada de errores técnicos.
    - *Lógica:* match en body_text o chat_transcripts buscando ("se cuelga", "crashea", "no carga", "crash") filtrado por la entidad extraída device_os = 'iOS' en las últimas 12 horas. Esto permite al equipo de IT saber si la última actualización de Apple ha roto la app.
5. *(Mi propuesta 2)* **Alerta de riesgo sanitario o intoxicación:**
    - *Tipo:* Búsqueda proactiva de alto riesgo.
    - *Lógica:* match_phrase o match de alta relevancia buscando ("intoxicación", "hospital", "enfermo", "vomito", "food poisoning") en reseñas y tickets, agrupado por restaurant_name. Si un restaurante tiene más de 2 menciones en una semana, genera una alerta automática para suspenderlo preventivamente e inspeccionarlo.

Con esta herramienta, el equipo de soporte deja de ir a ciegas y la dirección puede detectar crisis antes de que lleguen a Twitter o a Sanidad.

Paso 10 - Seguridad (matriz de riesgos + respuesta a incidentes)

Como nos demostró el reciente incidente del ex-empleado del equipo de BI, la seguridad no es un mero trámite de IT; es la supervivencia de UrbanEats. He diseñado esta estrategia de seguridad asegurándome de cubrir el ciclo completo: prevención (matriz y controles) y reacción (plan de incidentes).

1. Matriz de Riesgos

He completado las 5 amenazas obligatorias y añadido una sexta crítica (exposición de secretos), integrando todos los controles exigidos (IAM/RBAC, cifrado, auditoría, segregación, gestión de secretos y offboarding).

Amenaza	**Vulnerabilidad**	**Activo**	**Impacto**	**Control (Técnico/Organizativo)**	**Evidencia/Log**
Fuga masiva (tipo incidente)	Descargas sin límite ni alertas.	Data Warehouse / Data Lake	Multa millonaria AEPD,	**Cifrado en reposo**, DLP (Data Loss Prevention) bloqueando descargas >1000 filas.	Logs de auditoría de BD y alertas del DLP.

Amenaza	Vulnerabilidad	Activo	Impacto	Control (Técnico/Organizativo)	Evidencia/Log
			pérdida de confianza.		
Ex-empleado con accesos activos	Proceso de *offboarding* manual y lento.	Toda la infraestructura.	Robo de datos o sabotaje (borrado de tablas).	**Revocación de accesos automatizada en offboarding** ligada al SSO de RRHH.	Logs de IAM (Identity and Access Management) y SSO.
Inyección SQL en panel restaurante	Sanitización deficiente de inputs en la web.	Base de datos relacional.	Manipulación de precios, acceso no autorizado a otros restaurantes.	**Segregación de entornos** (VPC separada), WAF (Web Application Firewall) y consultas parametrizadas.	Logs del WAF y registros de error del motor SQL.
Suplantación de rider (GPS falso)	API de telemetría acepta datos sin validar la app/dispositivo.	Almacén operacional (active_riders).	Fraude en pagos, asignaciones erróneas, riesgo al cliente.	Autenticación robusta (tokens rotatorios), validación de integridad de la App y reglas lógicas de velocidad.	Logs de sesión de API Gateway y alertas de velocidad anómala.
Acceso no autorizado a direcciones	Permisos excesivos a nivel general y datos en claro.	Colección active_orders / Data Lake.	Riesgo físico para clientes, infracción grave RGPD.	**Cifrado en tránsito** (TLS) y estricto **IAM/RBAC** (agentes solo ven direcciones de sus casos asignados).	Logs de acceso a nivel de fila/documento en BD.
(Mi adición) Exposición de credenciales	*Hardcoding* de contraseñas de BD en el código fuente.	Repositorio GitHub.	Toma de control total del ecosistema de datos por atacantes externos.	**Gestión de secretos** centralizada (ej. HashiCorp Vault / AWS Secrets Manager).	Alertas de escaneo de código pre-commit (ej. GitLeaks).

2. Plan de Respuesta a Incidentes (IRP)

Si volvemos a sufrir una fuga como la de hace 2 meses, nuestro equipo de operaciones y legal ejecutará este plan de 4 fases:

- **Fase 1: Detección**
  - *Cómo nos enteramos:* No esperaremos a verlo en un foro. El sistema DLP y las alertas de comportamiento anómalo (UEBA) detectarán si un usuario de BI, por ejemplo, intenta hacer un SELECT * de la tabla de pedidos y exportarlo, disparando una alerta crítica al SOC en tiempo real.
- **Fase 2: Contención**
  - *Acción inmediata:* Cortar la hemorragia. Aislaremos la red del Data Warehouse, bloquearemos la IP sospechosa, revocaremos inmediatamente el token de sesión y los roles IAM del empleado o ex-empleado implicado, y bloquearemos los repositorios si fuera necesario.
- **Fase 3: Preservación de evidencias**
  - *Cadena de custodia:* Tomaremos un *snapshot* (copia exacta inmutable) del estado de la base de datos y de los logs de acceso (audit logs). Generaremos un *hash* criptográfico de estos logs para demostrar ante la AEPD y ante un juez que la evidencia no ha sido manipulada por nosotros tras el incidente.
- **Fase 4: Comunicación**
  - *Transparencia y legalidad:* Notificaremos a la AEPD en el plazo máximo legal de 72 horas. Inmediatamente después, el CEO enviará un comunicado honesto a los usuarios afectados (informando qué se filtró y qué medidas hemos tomado) para intentar salvar la reputación de la marca, además de informar al consejo de administración y gestionar el control de daños con la prensa.

Con esta matriz, dejamos de ser reactivos y empezamos a construir resiliencia.

Paso 11 - Privacidad (PII + minimización + retención)

Para recuperar la confianza de la AEPD y de nuestros usuarios, he diseñado esta estrategia de privacidad aplicando el principio de "Privacy by Design". No se trata solo de cumplir la ley, sino de reducir nuestra superficie de exposición al mínimo viable operativo.

Aquí presento el marco de privacidad para UrbanEats:

1. Inventario de PII y Base Legal RGPD

He clasificado todos los datos personales de nuestro ecosistema y les he asignado su base legal de tratamiento:

- **Identificadores Directos:** Nombres completos, direcciones exactas de entrega y números de teléfono (presentes en orders.csv y como PII accidental en support_tickets.json, chat_transcripts.txt y reviews.json).
    - *Base Legal:* **Ejecución de contrato** (imprescindible para llevar la comida al cliente y darle soporte).
- **Identificadores Indirectos:** customer_id, rider_id, geolocalización continua (lat, lon en rider_telemetry.jsonl).
    - *Base Legal:* **Ejecución de contrato** (para el enrutamiento) e **Interés Legítimo** (para optimizar los tiempos de asignación y detectar fraudes).
- **Datos Sensibles Inferibles (Art. 9 RGPD):** Las preferencias alimentarias (ej. restricciones por religión como Halal/Kosher o alergias médicas) que se pueden deducir cruzando el cuisine_type o los comentarios con la delivery_address.
    - *Base Legal:* El artículo 9 exige **Consentimiento Explícito** para tratar datos que revelen creencias religiosas o datos de salud. Como no lo pedimos explícitamente y fue lo que causó la fuga masiva, mi decisión de negocio es **no tratar este cruce con fines analíticos**, aplicando minimización radical en el Data Warehouse.

2. Técnicas de Protección: Pseudonimización vs. Anonimización

Para proteger estos datos en nuestras capas de almacenamiento, aplico dos enfoques distintos según la necesidad funcional:

- **Pseudonimización (Reversible):** La uso para los rider_id y customer_id en la capa *Curated* del Data Lake. Consiste en sustituir el ID real por un token o *hash* (ej. rider_A9F2). Existe una "tabla de correspondencia" fuertemente cifrada y aislada en otra base de datos, accesible solo por personal autorizado si necesitamos reidentificar a un usuario por un requerimiento legal.
- **Anonimización (Irreversible):** La uso para los textos de reviews.json o chat_transcripts.txt que van al motor de búsqueda para análisis de sentimiento. Técnicas de NLP (DLP) eliminan nombres o teléfonos. No hay forma técnica de volver atrás y saber quién escribió la reseña una vez anonimizada.
- **Tratamiento de Direcciones:** La dirección exacta es radiactiva. En el almacén operacional se usa en texto claro para que el rider llegue al destino. Sin embargo, en la ingesta hacia el Data Lake/Warehouse, la transformamos inmediatamente a un polígono geográfico (ej. cuadrícula H3 de Uber) o simplemente al código postal.

3. Política de Retención

No podemos guardar los datos para siempre. He definido estos ciclos de vida:

1. **Telemetría de riders (rider_telemetry.jsonl):** Retención de **30 días**. Tiempo suficiente para resolver disputas de pago por kilometraje o quejas de conducción temeraria. Luego, se purga.

2. **Direcciones exactas de entrega:** Retención de **90 días** tras el último pedido. Cubre sobradamente la ventana de devoluciones o reclamaciones de consumo. Pasado este tiempo, se trunca al código postal.
3. **Pedidos y Pagos agregados (orders.csv, payments.csv):** Retención de **4 a 5 años** por obligación fiscal y contable en España, pero manteniendo solo datos pseudonimizados sin items_json.
4. **Reseñas y Tickets de soporte:** Retención **indefinida, pero solo en su forma anonimizada**. Nos sirve para entrenar modelos de IA de atención al cliente sin violar la privacidad.

4. Minimización (El valor de decir "no")

Mi postura ante el CEO es clara: dato que no tenemos, dato que no nos pueden robar.

- **¿Necesitamos la dirección exacta después de la entrega?** No. Para los mapas de calor de demanda nos basta el barrio o zona.
- Por tanto, la instrucción arquitectónica es **dejar de propagar** la dirección completa y el detalle exacto del menú (items_json) más allá del almacén operacional temporal.

Con esto blindamos la empresa ante futuras auditorías.

Paso 12 - Mini-PIA (Evaluación de Impacto en Privacidad)

Como arquitecto de este nuevo ecosistema, y tras el expediente abierto por la AEPD debido a la fuga masiva, realizar esta Evaluación de Impacto en Privacidad (PIA) no es un mero trámite burocrático, es nuestra hoja de ruta para que el CEO pueda dormir tranquilo.

He desarrollado los 6 apartados exigidos evaluando críticamente lo que hacemos con los datos:

1. Tratamientos y finalidades En UrbanEats realizamos operaciones críticas con datos personales:

- **Geolocalización de riders:** Para seguimiento en vivo y cálculo de ETA.
- **Asignación automática de riders:** Algoritmo de emparejamiento pedido-repartidor.
- **Almacenamiento de direcciones exactas:** Necesario para entregar el pedido.
- **Análisis de sentimiento de reseñas:** Para medir la satisfacción y calidad.
- **Perfilado de consumo e inferencia de hábitos alimentarios:** Cruzar tipo de cocina con direcciones para marketing.

2. Necesidad y proporcionalidad

- ¿Es necesario geolocalizar al rider cada 3 segundos? Sí, operativamente para el algoritmo de asignación en tiempo real. Sin embargo, almacenarlo históricamente a esa granularidad es desproporcionado.

- ¿Necesitamos la dirección exacta siempre? Solo durante el reparto. Para analítica posterior, bastaría con el código postal o la zona.
- ¿Es necesario inferir hábitos alimentarios? Rotundamente no. El riesgo supera cualquier beneficio comercial.

3. Riesgos para los interesados He identificado 5 riesgos principales de nuestra operativa:

1. **Vigilancia de riders:** Control exhaustivo y estrés laboral por monitorización GPS continua.
2. **Perfilado de hábitos alimentarios:** Riesgo de inferir religión (ej. comida Halal) o condiciones de salud (alergias) , lo cual entra en la categoría de datos especialmente protegidos (Art. 9 RGPD).
3. **Reidentificación por dirección:** Que una base de datos anonimizada se cruce con otra pública y exponga a los clientes.
4. **Fuga masiva:** Repetición del incidente por acceso indebido interno o hackeo externo.
5. **Discriminación por barrio/zona:** Que nuestro algoritmo decida no enviar riders a ciertas zonas, afectando a usuarios por su código postal.

4. Salvaguardas Para cada riesgo, aplico estas contramedidas:

1. cite_start: Purga de la telemetría cruda a los 30 días. Agregación a nivel de zona para históricos.
2. cite_start: Eliminación del campo items_json del Data Warehouse analítico. Prohibición técnica de cruzar cuisine_type con delivery_address.
3. cite_start: Truncado de direcciones a Código Postal a los 90 días.
4. cite_start: IAM estricto, revocación automática de accesos, cifrado de bases de datos y herramientas DLP.
5. (Discriminación): Auditorías periódicas al algoritmo de asignación para garantizar equidad.

5. Riesgo residual Tras aplicar las salvaguardas, el riesgo de fuga masiva con impacto catastrófico se reduce drásticamente. El riesgo residual es **aceptable** operativamente. No podemos eliminar el 100% del riesgo de un hackeo, pero sí hemos eliminado el riesgo de exponer datos de salud o religión (porque ya no los procesamos para analítica).

6. Decisión **Se puede proceder con la nueva arquitectura**. *Decisión clave para el negocio:* Como hemos aplicado Privacidad por Diseño (eliminando la inferencia de datos sensibles del Art. 9 RGPD y truncando direcciones) , **NO necesitamos realizar una consulta previa a la AEPD**. Esto nos ahorra meses de bloqueo legal y demuestra al CEO que el nuevo diseño no solo es seguro, sino que acelera el negocio.

Paso 13 - Legal, propiedad intelectual y scraping

He definido la postura legal de UrbanEats en torno a 10-15 líneas ejecutivas para proteger a la compañía de demandas y blindar nuestros activos intangibles:

1. Datos de terceros y límites del scraping : Utilizamos Open Data municipal y registros sanitarios bajo licencias abiertas públicas, pero las reseñas de Google/TripAdvisor y las tarifas de Glovo/Uber Eats están protegidas por sus términos de servicio. Sí, Glovo podría intentar demandarnos por extraer sus tarifas alegando "competencia desleal" o sobrecarga de sus servidores. Para mitigarlo legal y éticamente, mi diseño de ingesta (Paso 4) es estrictamente no intrusivo: leemos y respetamos el robots.txt, espaciamos las peticiones para no saturar su red (*politeness*), y no extraemos datos personales, limitándonos a información pública de mercado.

2. Propiedad Intelectual (IP): Nuestro mayor valor (nuestra IP) reside en el algoritmo de asignación de riders en tiempo real, los modelos de predicción de demanda y los datasets curados de la capa *Curated Zone*. Estos activos se protegerán como secreto comercial mediante cláusulas de cesión de derechos en todos los contratos de empleados y proveedores. Respecto a los modelos entrenados con el comportamiento de los usuarios, al haber aplicado anonimización y descartado la PII (como la dirección exacta y el detalle del pedido) desde el diseño, la IP del modelo nos pertenece íntegramente y evitamos que un cliente ejerza su derecho de supresión (RGPD) exigiendo que destruyamos el algoritmo entero.

Paso 14 - KPIs y visualización mínima

Para que el CEO y los directores de área puedan pilotar UrbanEats sin perderse en detalles técnicos, he definido un panel de control (dashboard) enfocado en rentabilidad, calidad del servicio y seguridad.

1. Definición de 6 KPIs Estratégicos y Operativos

Aquí detallo los 6 indicadores clave, diseñados para disparar acciones concretas cuando algo va mal:

1. **Tiempo Medio de Entrega (TME)**
    - *Fórmula:* AVG(delivery_ts - order_ts) expresado en minutos.
    - *Fuente de datos:* orders.csv (consultado desde la tabla fact_orders del Data Warehouse).
    - *Frecuencia:* Horaria (para operaciones) y Diaria (para dirección).
    - *Umbral de alerta:* > 35 minutos de media en una zona (dispara alerta para incentivar a más riders a conectarse).

2. **Tasa de Rotura de Cadena de Frío**
    - *Fórmula:* (Pedidos con temp_celsius fuera de rango legal / Total de pedidos entregados) * 100.
    - *Fuente de datos:* Streaming de bag_sensors.jsonl cruzado con active_orders.
    - *Frecuencia:* Tiempo real (evaluado cada minuto).
    - *Umbral de alerta:* > 2% de los pedidos activos (alerta automática al equipo de calidad y al rider).

3. **Rentabilidad (Ingresos y Comisiones) por Zona**
    - *Fórmula:* SUM(commission_amount) agrupado por city_id y zone.
    - *Fuente de datos:* payments.csv y orders.csv (Data Warehouse).
    - *Frecuencia:* Batch diario (nocturno).
    - *Umbral de alerta:* Caída de ingresos > 10% respecto al mismo día de la semana anterior.

4. **Tasa de Cancelación / Fricción en App**
    - *Fórmula:* (Eventos tipo 'cancel' o 'error' en checkout / Total eventos 'search') * 100.
    - *Fuente de datos:* app_events.jsonl (vía capa Near real-time).
    - *Frecuencia:* Near real-time (cada 5 mins).
    - *Umbral de alerta:* Picos de errores > 5% (indica posible caída de la pasarela de pago o bug en la app).

5. **Utilización de Riders (Eficiencia Logística)**
    - *Fórmula:* (Riders con pedido asignado / Total de Riders conectados) * 100.
    - *Fuente de datos:* Colección NoSQL active_riders.
    - *Frecuencia:* Tiempo real.
    - *Umbral de alerta:* > 90% (saturación, los clientes sufrirán retrasos) o < 40% (riders inactivos perdiendo dinero).

6. **Bloqueos de Seguridad / Privacidad (El KPI del CEO)**
    - *Fórmula:* Conteo absoluto de consultas masivas bloqueadas por el DLP o accesos no autorizados.
    - *Fuente de datos:* Logs del sistema IAM y reglas del Data Lake.
    - *Frecuencia:* Tiempo real.
    - *Umbral de alerta:* > 0. Tolerancia cero. Un solo bloqueo dispara el Plan de Respuesta a Incidentes (Fase 2: Contención).

---

2. Visualizaciones Propuestas (Mockups)

Para que estos datos sean consumibles de un vistazo, propongo estas dos visualizaciones en nuestra herramienta de BI:

Visualización A: Mapa de Calor Operacional (Demanda vs. Oferta)

- *Descripción:* Un mapa interactivo de la ciudad (ej. Madrid). Muestra zonas pintadas en rojo intenso donde hay alta concentración de pedidos pendientes, superpuesto con puntos verdes que representan a los riders actualmente "Disponibles".
- *Datos que la alimentan:* Coordenadas de la colección active_riders (telemetría a 3s) y las coordenadas ofuscadas (centroide de la zona postal) de los pedidos en active_orders.
- *Valor de negocio:* Permite al equipo de operaciones aplicar "surge pricing" (tarifas dinámicas) o enviar notificaciones push a los riders para que se desplacen a las zonas rojas, equilibrando la oferta y la demanda.

Visualización B: Funnel de Conversión de la App

- *Descripción:* Un gráfico de embudo clásico que muestra la caída de usuarios en cada paso: Búsqueda (search) ➔ Añadir a carrito (add_cart) ➔ Pago (checkout) ➔ Entregado (delivered).
- *Datos que la alimentan:* Eventos procesados de app_events.jsonl y estados finales de orders.csv.
- *Valor de negocio:* Si vemos que 10.000 usuarios buscan comida pero solo 2.000 llegan al carrito, sabemos exactamente dónde está fallando la experiencia de usuario (UX) o si los precios de los restaurantes están espantando a los clientes.

Con esto, hemos completado todos los pasos técnicos y estratégicos del caso.

Paso 15 - DOSSIER FINAL: Las 15 Preguntas (El Pitch al CEO)

Aquí presento mi dossier final con las 15 respuestas numeradas, sintetizando todo el trabajo técnico y estratégico. Mi enfoque principal ha sido reconstruir la confianza de UrbanEats, garantizando que el negocio escale sin volver a enfrentarnos a una multa de la AEPD.

Pregunta 1: ¿Cuáles son TODAS las fuentes del caso y cómo se clasifican? Tenemos 13 fuentes. Se clasifican en:

- **Estructuradas (Batch):** orders.csv, payments.csv, restaurants.csv, riders_shifts.csv (Propietarias) y Open Data municipal (Tercero).
- **Semiestructuradas (Streaming/Near RT):** app_events.jsonl, rider_telemetry.jsonl (contiene PII), bag_sensors.jsonl (Propietarias).
- **No estructuradas:** support_tickets.json, chat_transcripts.txt, reviews.json (Propietarias/Tercero, Near RT/Batch, con altísima sensibilidad por PII accidental). Webs de competidores y Registros sanitarios (Scraping Batch, Terceros, sin PII).

Pregunta 2: ¿Qué datos DESCARTAS y por qué? Descarto propagar el campo items_json (el detalle del pedido) hacia el Data Warehouse analítico. *Por qué:* Cruzarlo con la dirección exacta generó la fuga de datos sensibles inferibles (religión/salud). Para calcular ingresos por tipo de cocina (lo que le importa a finanzas), no necesitamos saber si el cliente pidió sin gluten. Descarto también la dirección exacta pasada la entrega, quedándome solo con el código postal para evitar sesgos y reidentificación.

Pregunta 3: Escribe tus 8+ reglas de calidad, cómo las medirías y acción.

1. **Unicidad:** COUNT(order_id) == 1 en orders.csv. Query SQL. Acción: Eliminar duplicados.
2. **Validez:** temp_celsius BETWEEN -5 AND 60. Filtro en streaming. Acción: Descartar (glitch de sensor).
3. **Exactitud:** GPS del rider dentro de la ciudad (city_id). Función geoespacial (ST_Contains). Acción: Descartar.
4. **Validez:** total_price >= 0. CHECK SQL. Acción: Cuarentena (alerta bug de cobro).
5. **Consistencia:** delivery_ts (UTC) > order_ts (UTC). Script ETL. Acción: Convertir a UTC y corregir.
6. **Integridad:** rider_id en telemetría existe en turnos activos. JOIN streaming-batch. Acción: Cuarentena (riesgo seguridad).
7. **Integridad:** restaurant_id en pedidos existe en catálogo. LEFT JOIN. Acción: Cuarentena.
8. **Privacidad:** Textos de soporte/chats sin PII (regex DNI/teléfonos). Script DLP. Acción: Corregir enmascarando ([PII_REMOVED]) antes de guardar.

Pregunta 4: Describe el pipeline end-to-end incluyendo scraping y streaming.

- **Ingesta:** Batch (ETL) para transacciones; Streaming dual vía Kafka (prioridad sub-segundo para rider_telemetry, paralela para bag_sensors); Scraping ético programado (con retardos y respeto a robots.txt) para webs externas. Aplicamos DLP en vuelo a los textos libres.
- **Raw Zone:** Aterrizaje inmutable en Data Lake (JSON/CSV/HTML crudo).
- **Curated Zone:** Normalización horaria, limpieza de nulos y conversión a formato columnar (Parquet) uniendo ventanas temporales de posición de rider y temperatura de bolsa.
- **Serving Zone:** Distribución a SQL (analítica), NoSQL (operativa) y motor de búsqueda (soporte).

Pregunta 5: Arquitectura (lake / warehouse / NoSQL / búsqueda) y por qué existen.

- **Data Lake:** Existe como almacén histórico, inmutable y auditable (Raw + Curated).
- **Warehouse (SQL):** Existe para la analítica (Dashboards del CEO). Almacena métricas agregadas libres de PII para consultas complejas (OLAP).
- **NoSQL:** Existe para la velocidad operativa (App). Almacena el estado efímero del pedido y rider para asignación en milisegundos (OLTP).
- **Búsqueda:** Existe para que soporte recupere tickets y reseñas instantáneamente usando índices de texto libre enmascarado.

Pregunta 6: CAP-A (asignación de rider): elección, por qué y mitigación. Elijo **AP (Disponibilidad)**. Si hay una caída de red parcial, la app debe seguir aceptando y asignando pedidos locales para no perder dinero frente a Glovo. El riesgo es asignar dos pedidos simultáneos al mismo rider (inconsistencia). Lo mitigo (buscando consistencia eventual) usando colas de reasignación y *leases* en la app del rider para que solo confirme el primer pedido.

Pregunta 7: CAP-B (dashboard financiero): elección, por qué y mitigación. Elijo **CP (Consistencia)**. Para las finanzas del CEO, es inaceptable tener ingresos duplicados o erróneos por un fallo de sincronización de red. Si hay un fallo, el dashboard falla (no está disponible). Lo garantizo usando procesos idempotentes y transacciones ACID en el modelo relacional.

Pregunta 8: Presenta tu modelo SQL y 2 KPIs con 2 consultas ejemplo. Modelo en Estrella: fact_orders vinculada a dim_date, dim_time, dim_location, y dim_restaurant.

1. *KPI: Ingresos por ciudad* -> SELECT l.city_name, SUM(f.total_price) FROM fact_orders f JOIN dim_location l ON f.city_id = l.city_id GROUP BY l.city_name;
2. *KPI: Tiempo de entrega por franja* -> SELECT t.time_band, AVG(f.delivery_time_mins) FROM fact_orders f JOIN dim_time t ON f.time_id = t.time_id GROUP BY t.time_band;

Pregunta 9: Presenta tu modelo NoSQL con 2 docs, índices y 2 consultas. *Docs:* Colección active_riders (contiene ID, ubicación geoespacial, batería, y bolsa anidada) y active_orders (contiene ID, rider asignado referenciado, y coords de zona ofuscadas). *Índices:* Geoespacial 2dsphere en current_location del rider e índice B-Tree en assigned_rider_id del pedido. *Consultas (vía JSON*

payload): (1) Filtro $near buscando status: "AVAILABLE" y limitando a 5 resultados. (2) Búsqueda del pedido por _id seguida de otra consulta al rider aplicando projection para traer solo la ubicación y la temperatura.

Pregunta 10: Índice de tickets/chat/reseñas y 5 consultas operativas. Índice unificado con tokenización, stemming y stopwords. Entidades extraídas: zona, cocina, SO. Consultas:

1. Top motivos de reclamación (Agregación Terms).
2. Búsqueda de "frío/cold" filtrado por cuisine_type = 'Sushi' y Madrid.
3. "Devolución" en texto filtrado por prioridad ALTA.
4. "Crashea" o "no carga" filtrado por device_os = 'iOS'.
5. "Intoxicación/Hospital" agrupado por restaurant_name para alertas de sanidad.

Pregunta 11: Matriz de riesgos: identifica 6 amenazas y sus controles.

1. *Fuga masiva:* Control DLP bloqueando select masivos.
2. *Ex-empleado:* Revocación de IAM vía automatización de offboarding.
3. *Inyección SQL:* Segregación de red y WAF.
4. *GPS Falso:* Validación de integridad de la API.
5. *Acceso direcciones:* Cifrado en reposo/tránsito y RBAC estricto.
6. *Claves expuestas:* Gestión de secretos centralizada (Vault). *(Respuesta a incidentes: Detección vía alertas, Contención aislando red, Preservación de logs, Comunicación a AEPD en 72h)*.

Pregunta 12: PII existente y medidas aplicadas (minimización, retención, pseudo/anon). Existe PII directa (nombres, direcciones), indirecta (GPS rider) y datos sensibles inferibles (comida+dirección = religión). *Medidas:* Minimización (eliminar items_json del analítico). Pseudonimización para IDs en el Data Lake. Anonimización vía NLP para textos de soporte. *Retención:* Telemetría 30 días, direcciones completas 90 días (luego código postal), histórico 5 años pseudonimizado.

Pregunta 13: Mini-PIA: riesgos, salvaguardas y riesgo residual. *Riesgos:* Vigilancia excesiva laboral, perfilado alimentario (Art. 9 RGPD), reidentificación. *Salvaguardas:* Purga de GPS a 30 días, eliminación física de la capacidad de cruzar pedidos al detalle con el hogar en la capa BI. *Riesgo residual:* Aceptable. No procesamos datos sensibles por diseño, evitando sanciones críticas y la necesidad de consulta previa a la AEPD.

Pregunta 14: Scraping y terceros: límites legales/éticos y cómo los cumples. Asumo el riesgo de que Glovo alegue competencia desleal o sobrecarga. Lo mitigo estrictamente leyendo su robots.txt, sin extraer PII, e insertando *delays* aleatorios (politeness) para no afectar sus servidores. Datos municipales son Open Data sin restricciones.

Pregunta 15: Propiedad intelectual: ¿qué genera IP y cómo la gestionas? El algoritmo de emparejamiento y los datasets limpios son la IP núcleo. Se licencia internamente como secreto

comercial. Al haber separado la PII de los datos de entrenamiento (anonimización por diseño), evito que si un usuario pide borrar sus datos (derecho al olvido RGPD), la empresa se vea obligada a reentrenar o destruir su IP principal.

DECLARACIÓN DE USO DE IA (Obligatoria)

1. Declaración de uso: He utilizado IA generativa como asistente para acelerar la ideación de la matriz de riesgos, validar la sintaxis de las consultas de agregación (Elasticsearch/SQL) y estructurar la redacción del dossier final. Las decisiones críticas de negocio, como la resolución de los subsistemas CAP, la decisión de descartar el campo items_json por motivos de privacidad, y el diseño del esquema en estrella, han sido generadas íntegramente mediante mi criterio profesional.

#	Prompt enviado	Salida útil recibida	Decisión humana
1	*"Genera una tabla comparando características del teorema CAP para sistemas de delivery."*	Lista teórica de CP vs AP.	Descarté la teoría pura y diseñé escenarios específicos de fallos de red (viernes noche) para priorizar el negocio.
2	*"¿Cómo estructurarías un pipeline que combine batch y streaming (Kafka)?"*	Arquitectura Lambda estándar.	Ajusté la respuesta para adaptarla a la *Curated Zone*, uniendo la temperatura y el GPS temporalmente.
3	*"Crea un JSON de ejemplo para el estado de un rider en MongoDB."*	JSON plano con todos los datos embebidos.	Decidí dividirlo en dos colecciones (active_riders y active_orders) usando referencias para evitar bloqueos por alta escritura.
4	*"Dame ideas de amenazas de ciberseguridad para una matriz de riesgos."*	Lista genérica de OWASP.	Seleccioné 5 más la "Exposición de secretos en GitHub" adaptándolo al offboarding manual del caso.
5	*"Resume los artículos 9 del RGPD relativos a datos de salud y religión."*	Explicación legal del consentimiento explícito.	Apliqué esta base para eliminar proactivamente el campo items_json de la capa analítica (Minimización/Privacy by Design).